杭州電子科技大学

本 科 毕 业 设 计

（2026届）

题　　目	基于 Spring Boot 与 Vue 3 的企业供应链协同管理系统的设计与实现
学　　院	计算机学院
专　　业	计算机科学与技术
班　　级	22052320
学　　号	22051010
学生姓名	廖波南
指导教师	李运发
完成日期	2026 年 5 月

诚 信 承 诺

我谨在此承诺：本人所写的毕业论文《基于 Spring Boot 与 Vue 3 的企业供应链协同管理系统的设计与实现》均系本人独立完成，没有抄袭行为，凡涉及其他作者的观点和材料，均作了注释，若有不实，后果由本人承担。

承诺人（签名）

年　　月　　日

摘　要

随着制造业数字化转型持续推进，企业内部订单处理、仓储管理、生产执行、采购协同与质量追溯等业务环节之间的联动关系日益紧密。传统以人工沟通和分散系统支撑为主的管理方式，普遍存在信息传递滞后、库存状态不透明、跨部门协同效率较低以及质量问题追踪链条不完整等不足。为提升供应链业务协同能力，本文围绕企业供应链场景，设计并实现了一套基于 Spring Boot 与 Vue 3 的企业供应链协同管理系统。系统采用前后端分离架构，以 Spring Security 与 JWT 完成身份认证和基于角色的权限控制，以 MySQL 与 JPA 实现业务数据持久化，并结合 WebSocket/STOMP 构建实时消息通知机制，从而支持顾客、销售管理员、仓库管理员、生产管理员、采购管理员、供应商、质检管理员以及系统管理员等多角色在统一平台上协同作业。

本文从系统需求分析、总体架构设计、业务流程设计、数据库设计以及关键功能实现等方面展开论述。系统围绕订单驱动主线，构建了销售审核、仓储发货、生产执行、采购供应、质检留痕与质量追溯的闭环业务流程，并在此基础上实现库存预警与周计划生成机制，以增强企业对库存风险和计划衔接问题的响应能力。论文重点分析了角色权限控制、订单至质检协同流程、采购至收货协同流程、实时消息推送、质量追溯以及周计划生成等关键实现机制。测试结果表明，所设计系统能够较好地满足企业供应链协同管理的基本需求，在流程贯通性、数据一致性和业务可追溯性方面具有较好的工程应用价值。

关键词：供应链协同管理；Spring Boot；Vue 3；库存预警；质量追溯

ABSTRACT

With the continuous advancement of digital transformation in manufacturing enterprises, the interactions among order processing, warehouse management, production execution, procurement collaboration and quality traceability have become increasingly close. Traditional management approaches supported by manual communication and isolated information systems often suffer from delayed information transmission, insufficient inventory visibility, low cross-department collaboration efficiency and incomplete quality tracking chains. To improve collaborative management capability in enterprise supply chains, this thesis designs and implements an enterprise supply chain collaborative management system based on Spring Boot and Vue 3. The system adopts a front-end and back-end separated architecture. Spring Security and JWT are used to implement identity authentication and role-based access control, MySQL and JPA are employed for business data persistence, and WebSocket/STOMP is introduced to support real-time message notification. In this way, multiple roles, including customers, sales administrators, warehouse administrators, production administrators, procurement administrators, suppliers, quality inspectors and system administrators, can collaborate on a unified platform.

This thesis discusses the system from the perspectives of requirement analysis, overall architecture design, business process design, database design and key function implementation. Centered on order-driven business coordination, the system establishes a closed-loop process covering sales review, warehouse delivery, production execution, procurement collaboration, quality inspection recording and quality traceability. On this basis, inventory warning and weekly plan generation mechanisms are implemented to enhance the enterprise's responsiveness to inventory risks and planning coordination problems. The thesis further analyzes representative implementation mechanisms, including role-based authorization, order-warehouse-production-quality coordination, procurement-supplier-warehouse collaboration, real-time message push, quality traceability and weekly planning. Test results indicate that the system can effectively satisfy the basic requirements of enterprise supply chain collaboration and demonstrates practical engineering value in process continuity, data consistency and business traceability.

Keywords: Supply Chain Collaboration; Spring Boot; Vue 3; Inventory Warning; Quality Traceability

目　录

1　绪论 1
1.1　选题背景、目的和意义 1
1.2　国内外研究现状 1
1.2.1　国外研究现状 1
1.2.2　国内研究现状 2
1.3　研究内容与论文结构安排 2
1.3.1　研究内容 2
1.3.2　论文结构安排 3
2　理论基础与相关技术 4
2.1　B/S 架构与前后端分离模式 4
2.2　Spring Boot 框架 4
2.3　Spring Security 与 JWT 身份认证机制 4
2.4　Vue 3、Vite 与 Axios 前端技术 5
2.5　WebSocket/STOMP 实时通信技术 5
2.6　MySQL 与 JPA 数据持久化技术 6
2.7　本章小结 6
3　系统分析与设计 7
3.1　系统概述 7
3.2　需求分析 7
3.2.1　用户角色分析 7
3.2.2　功能需求分析 8
3.2.3　非功能需求分析 8
3.3　整体架构设计 8
3.3.1　系统总体架构 8
3.3.2　前后端分层设计 9
3.3.3　权限控制设计 9
3.4　业务流程设计 9
3.4.1　客户下单与销售审核流程 9
3.4.2　仓储发货与库存流转流程 10
3.4.3　生产计划、生产执行与质检流程 10
3.4.4　原材料采购与供应商协同流程 10
3.4.5　库存预警与周计划生成流程 10

3.4.6 质量追溯流程 10
3.5 数据库设计 11
3.5.1 概念结构设计 11
3.5.2 主要数据表设计 11
3.5.3 关键实体关系分析 11
4 系统详细设计与实现 13
4.1 开发环境与工具 13
4.2 各功能模块实现 13
4.2.1 用户认证与账号管理模块实现 13
4.2.2 顾客下单与订单管理模块实现 13
4.2.3 销售管理模块实现 14
4.2.4 仓库管理与库存预警模块实现 14
4.2.5 生产管理模块实现 14
4.2.6 采购与供应商管理模块实现 15
4.2.7 质检与质量追溯模块实现 15
4.2.8 实时消息通知模块实现 15
4.2.9 周计划智能生成模块实现 15
5 核心功能实现 17
5.1 基于角色的访问控制与身份认证实现 17
5.2 订单—仓储—生产—质检协同流程实现 17
5.3 采购—供应商—仓库协同流程实现 17
5.4 库存预警与周计划自动生成算法实现 18
5.5 实时消息推送与状态同步实现 19
5.6 质量追溯功能实现 19
6 系统测试 21
6.1 测试环境与工具 21
6.2 单元测试 21
6.3 功能测试 22
6.4 测试结果分析 22
7 总结与展望 23
7.1 工作总结 23
7.2 研究不足与未来展望 23
致谢 24
参考文献 25

1 绪论

1.1 选题背景、目的和意义

当前制造业企业正处于由单点信息化向全流程数字化协同转型的关键阶段。订单获取、库存控制、生产执行、原材料采购以及质量管理等环节相互耦合，任何单一节点的信息延迟或处理失误，都会沿供应链链路放大并传导至交付周期、库存成本和客户满意度等核心指标。尤其是在多角色共同参与的企业业务场景下，如果订单状态、库存数量、采购进度和质检结果无法实现统一管理与实时共享，则容易形成信息孤岛，造成重复录入、人工催办、库存积压和追溯困难等问题。因此，构建一套面向企业内部多部门协同和上下游业务联动的供应链协同管理系统，已成为提升企业运营效率和管理精细化水平的重要途径[1, 2]。

从企业管理视角看，供应链管理系统已不再局限于订单记录和库存统计，而是逐步发展为集订单驱动、库存联动、生产计划、采购协同、质量追溯和消息通知于一体的综合业务平台。系统建设的重点不只是实现基础数据电子化，还在于建立跨岗位、跨流程、跨状态的协同机制，使顾客、销售管理员、仓库管理员、生产管理员、采购管理员、供应商、质检管理员和系统管理员等角色能够围绕统一数据源开展工作，并通过可控的权限边界实现职责分离与业务闭环[3, 4]。

基于上述背景，本文以企业供应链协同管理为研究对象，设计并实现了一套基于 Spring Boot 与 Vue 3 的企业供应链协同管理系统。系统采用前后端分离架构，在后端使用 Spring Boot 构建 RESTful 服务，通过 Spring Security 与 JWT 实现用户认证与授权控制，采用 MySQL 与 JPA 完成业务数据管理，并利用 WebSocket/STOMP 支持关键业务节点的实时消息推送。本文的研究意义主要体现在以下三个方面：其一，从业务层面提升订单、库存、生产、采购和质检之间的流程衔接效率；其二，从系统层面增强多角色并发访问环境下的数据一致性、可追溯性和可扩展性；其三，从工程实践层面为中小型制造企业构建轻量化、易维护的供应链协同平台提供可参考的设计思路。

1.2 国内外研究现状

1.2.1 国外研究现状

国外关于供应链管理的研究起步较早，研究内容已从单纯的物流优化扩展到面向供应链韧性、协同决策、库存配置和质量追踪的系统化管理框架。Christopher 从物流与供应链管理一体化视角强调了企业通过信息共享和流程整合提升响应能力的重要性[1]。Lee 提出的“三 A 供应链”思想进一步指出，敏捷性、适应性与协同性是现代供应链系统建设的重要目标[2]。在企业信息系统层面，ERP、MES 与 SCM 等系统逐渐形成互补

关系，其中 ERP 更强调资源计划与财务控制，MES 更关注车间执行，而 SCM 系统则更注重跨部门和跨主体的业务协同。随着 Web 技术和服务架构的发展，面向浏览器访问、支持分布式协同的业务系统已成为主流实现方式。

在系统实现方法方面，国外研究和工业实践较多采用分层架构、无状态认证、消息驱动协同和服务接口标准化等手段提升系统的可维护性与可扩展性。针对质量追溯场景，国外企业普遍重视关键业务节点数据留痕，通过批次标识、状态记录和操作日志实现问题定位与责任界定。这些研究和实践为本文开展企业供应链协同管理系统的设计与实现提供了重要参考。

1.2.2 国内研究现状

国内在企业信息化和制造业数字化转型推动下，围绕供应链管理、库存控制、采购协同、质量追溯和制造执行的研究不断深化。现有研究普遍认为，传统企业在信息系统建设过程中存在系统割裂、角色边界不清晰、业务状态难以联动以及质量追溯链条不完整等问题，因此需要面向业务闭环重新整合订单、仓储、生产、采购和质检数据。与此同时，随着国家标准与行业规范不断完善，质量追溯体系建设受到越来越多关注，可追溯性已成为企业提升质量透明度和客户信任度的重要支撑[5]。

从实现路径看，国内相关系统研究更加注重工程可行性与业务适配性。一类研究侧重于 ERP、WMS、MES 等现有系统的功能扩展与对接，另一类研究则采用 Spring Boot、Vue 等主流 Web 技术重构业务流程，以实现更灵活的前后端分离与角色化访问控制。总体而言，现有研究为供应链协同系统建设提供了较好基础，但针对多角色权限隔离、订单驱动下的仓储与生产联动、库存预警触发采购协同、质检留痕与质量追溯、实时消息同步等方面，仍需结合具体业务流程开展更细致的工程设计与实现分析。

1.3 研究内容与论文结构安排

1.3.1 研究内容

围绕企业供应链协同场景，本文的主要研究内容包括以下几个方面。

1. 分析系统所面向的业务环境与用户角色，明确顾客、销售管理员、仓库管理员、生产管理员、采购管理员、供应商、质检管理员和系统管理员在业务协同中的职责边界与功能需求。

2. 设计系统总体架构、前后端分层方式、权限控制机制以及数据库结构，构建面向订单驱动的业务闭环，实现订单、库存、生产、采购、质检与追溯数据的统一管理。

3. 实现用户认证与账号管理、顾客下单与订单管理、销售审核、仓库管理与库存预警、生产管理、采购与供应商管理、质检与质量追溯、实时消息通知以及周计划生成等核心模块。

4. 重点研究基于角色的访问控制、订单至质检协同流程、采购供应协同流程、库存预警与周计划联动机制、实时消息推送以及质量追溯的工程实现方法，并通过系统测试验证其可用性与合理性。

1.3.2 论文结构安排

本文共分为七章，各章内容安排如下。

1. 第 1 章为绪论，主要阐述课题背景、研究目的与意义，总结国内外相关研究现状，并说明本文的研究内容与整体结构安排。

2. 第 2 章为理论基础与相关技术，介绍 B/S 架构、前后端分离模式、Spring Boot、Spring Security、JWT、Vue 3、Vite、Axios、WebSocket/STOMP、MySQL 与 JPA 等关键技术，并分析其在本系统中的适用性。

3. 第 3 章为系统分析与设计，从需求分析、总体架构、权限控制、业务流程以及数据库设计等方面完成系统的整体设计。

4. 第 4 章为系统详细设计与实现，围绕各业务模块的实现过程，分析前端页面、后端接口、业务服务与数据实体之间的协同关系。

5. 第 5 章为核心功能实现，重点讨论基于角色的访问控制、业务闭环协同、库存预警与周计划生成、实时消息同步以及质量追溯等关键实现机制。

6. 第 6 章为系统测试，介绍测试环境、单元测试与功能测试设计，并对测试结果进行分析。

7. 第 7 章为总结与展望，对全文工作进行总结，分析系统存在的不足，并提出后续优化方向。

2 理论基础与相关技术

2.1 B/S 架构与前后端分离模式

B/S 架构以浏览器作为统一访问入口，服务器端承担主要业务处理与数据管理职责。相较于传统 C/S 模式，B/S 架构在部署维护、跨平台访问以及多角色共享统一业务入口方面具有明显优势。对于企业供应链协同场景而言，顾客、内部管理人员与供应商往往处于不同终端和不同网络环境中，若系统需要支持灵活访问、集中维护和快速迭代，则 B/S 架构更符合实际业务需求。

前后端分离模式是在 B/S 架构基础上的进一步演进，其核心思想是将界面呈现与业务逻辑解耦，前端负责页面渲染、交互控制与状态展示，后端负责认证鉴权、业务处理、数据访问与规则校验。该模式有助于提高系统的开发效率和模块复用能力，也便于后续移动端、小程序端或第三方系统调用统一接口。结合供应链协同系统多角色、多流程、状态变更频繁的特点，前后端分离模式能够有效降低耦合度，为权限控制、消息推送和业务扩展提供良好的结构基础[6]。

2.2 Spring Boot 框架

Spring Boot 是基于 Spring 生态的快速开发框架，通过自动配置、约定优于配置、嵌入式容器支持以及完善的组件集成能力，简化了企业级 Web 应用的开发过程[7]。在系统实现中，开发者可围绕控制层、服务层、持久层进行分层组织，并借助统一配置文件管理安全、数据库和消息等相关参数。

对于本文设计的供应链协同管理系统而言，Spring Boot 的优势主要体现在三个方面。其一，系统业务模块较多，包括订单、库存、生产、采购、质检和消息通知等，Spring Boot 便于快速构建结构清晰的 RESTful 后端服务。其二，Spring Boot 与 Spring Security、Spring Data JPA 等组件具有良好的集成性，能够降低认证授权和数据访问模块的实现复杂度。其三，系统后期可能存在功能增量开发需求，例如报表分析、移动端接口扩展等，Spring Boot 具备较好的可维护性与扩展能力，因此适合作为本系统的后端开发框架。

2.3 Spring Security 与 JWT 身份认证机制

在多角色业务系统中，身份认证与权限控制是保障数据安全和职责边界清晰的基础。Spring Security 提供了较为完善的认证授权机制，可通过过滤器链、用户详情服务、密码编码器以及访问控制表达式等方式实现访问拦截和资源保护[8]。JWT 是一种基于 JSON 的令牌规范，通过对用户身份与权限声明进行签名封装，使服务器在后续请求处理中能够基于令牌完成无状态身份校验[9]。

本系统采用 Spring Security 与 JWT 相结合的方式实现认证授权。用户登录成功后，服务器生成包含用户标识和角色信息的令牌，并返回给前端保存。前端在后续接口调用时将令牌附加到请求头中，后端过滤器解析令牌后完成用户身份恢复和权限上下文构建。该方式具有以下适用性：一是能够降低传统 Session 模式下的服务器状态维护压力，适合前后端分离系统；二是便于结合角色信息实现基于路径和方法级别的访问控制；三是有利于在订单审核、仓库出入库、采购审批和质检处理等高敏感操作中进行统一鉴权。

2.4 Vue 3、Vite 与 Axios 前端技术

Vue 3 是面向构建用户界面的渐进式前端框架，具备组件化开发、响应式数据驱动和良好的生态支持等特征[10]。在企业供应链管理场景中，前端页面通常涉及表单录入、列表查询、状态过滤、流程操作和实时消息展示等复杂交互，Vue 3 的组件化机制能够将订单表单、库存表格、生产任务面板、采购处理页面和消息中心等视图进行模块化组织，提升前端代码复用性和维护效率。

Vite 是现代前端构建工具，依托原生 ES 模块和快速热更新机制，在开发体验和构建效率方面具有明显优势[11]。Axios 则用于实现前端与后端之间的 HTTP 通信，能够统一封装请求头、响应拦截、异常处理和令牌附带逻辑。对于本系统而言，Vue 3、Vite 与 Axios 的组合不仅有利于构建响应迅速、交互明确的业务界面，也便于统一管理多角色登录态、接口异常与业务操作反馈，从而提升前端实现的规范性和一致性。

2.5 WebSocket/STOMP 实时通信技术

供应链协同系统中的多个业务节点具有明显的时效性要求。例如，销售审核结果需要及时反馈给仓库，库存不足状态需要尽快触发生产或采购处理，采购订单下发后供应商需要及时获知，质检结果也需要同步给相关岗位。若完全依赖前端轮询获取状态变化，不仅增加服务器负载，也容易带来消息延迟。WebSocket 通过在客户端与服务器之间建立持久连接，实现双向实时通信，适合处理频繁状态变化场景[12]。STOMP 则在 WebSocket 之上提供了轻量级消息协议，有助于实现基于主题或队列的消息路由与订阅机制[13]。

本系统基于 WebSocket/STOMP 构建实时消息通知模块。后端在订单审核、库存预警、采购下发、生产任务生成和质检状态更新等关键节点发布消息，前端通过订阅相应主题获取通知并更新界面状态。该技术方案有助于减少人工催办成本，提高跨角色协同的及时性，并使系统在业务流程驱动下具备更好的状态同步能力。

2.6 MySQL 与 JPA 数据持久化技术

供应链协同管理涉及用户、角色、产品、订单、库存、生产计划、采购订单、原材料、质检记录和消息通知等大量结构化数据，系统需要在保证数据一致性的前提下完成高频读写和关联查询。MySQL 作为成熟的关系型数据库，具有较好的稳定性、事务支持能力和生态兼容性，适合承载中小型企业业务系统的核心数据[14]。

JPA 提供了面向对象的数据持久化规范，使开发者能够通过实体映射方式管理数据库表结构与对象关系。基于 Spring Data JPA，可进一步简化常见的增删改查、条件查询与分页访问实现[15]。在本系统中，JPA 的适用性主要体现在三个方面：其一，用户、订单、库存、采购和质检等实体之间关联关系明确，适合采用实体映射建模；其二，系统需要在业务服务层组织统一事务，以确保状态流转和数据更新的一致性；其三，JPA 有助于减少重复的数据访问代码，提高后端开发效率。

2.7 本章小结

本章围绕系统设计与实现所依赖的关键技术进行了分析。B/S 架构与前后端分离模式为多角色统一访问和系统扩展提供了总体支撑；Spring Boot 为后端服务构建提供了高效框架；Spring Security 与 JWT 满足了系统对认证授权和角色控制的需求；Vue 3、Vite 与 Axios 适合构建复杂业务界面；WebSocket/STOMP 支撑了实时消息通知；MySQL 与 JPA 则为业务数据持久化提供了可靠基础。上述技术共同构成了本文企业供应链协同管理系统的技术实现基础。

3 系统分析与设计

3.1 系统概述

本文设计的企业供应链协同管理系统面向典型制造企业的内部协同与上下游联动场景，围绕订单驱动主线，对销售审核、库存流转、生产执行、采购供应、质量检验与追溯等关键业务进行统一组织。系统采用前后端分离架构，以角色化访问控制为约束，以状态流转为业务驱动，以实时通知和记录留痕为协同支撑，目标是在统一平台内实现跨岗位业务处理和关键数据共享。

从业务定位看，系统并非单一的仓储管理工具或采购登记工具，而是强调订单、库存、生产、采购和质检之间的闭环衔接。顾客发起订单后，系统通过销售审核、仓库处理、生产计划、质检入库和发货执行形成连续流程；原材料短缺时，系统则通过库存预警和采购申请联动采购管理员与供应商，保障生产执行的连续性。由此，系统既关注业务流程效率，也关注过程可追溯与职责边界控制。

3.2 需求分析

3.2.1 用户角色分析

本系统面向多角色协同场景，不同角色的业务职责、可访问资源和操作边界存在显著差异。若系统不能根据角色划分权限范围，容易引发数据越权访问和职责混淆问题。因此，在需求分析阶段需首先明确各类用户角色及其核心职责。

表 3-1 系统主要用户角色及职责

角色	核心目标	主要职责
系统管理员	维护系统运行秩序	维护用户账号、角色分配、基础数据与权限配置
顾客	提交业务需求与查询结果	注册登录、下达订单、查看订单处理状态、进行质量追溯查询
销售管理员	审核客户订单	审核订单有效性、维护销售记录、推动订单进入仓储处理环节
仓库管理员	管理库存与出入库	维护成品与原材料库存、处理出入库、执行发货、触发库存预警响应
生产管理员	组织生产执行	接收生产计划、下发生产任务、维护生产记录、协调质检前置环节
采购管理员	保障物料供应	审核采购申请、生成采购订单、跟踪供应商执行与到货情况
供应商	响应采购协同	查看采购订单、确认接单、执行发货并反馈发货状态
质检管理员	控制质量过程	处理来料或成品质检、登记质检结果、支撑质量追溯

由表 3-1可见，本系统的设计难点不在于单一功能点的实现，而在于不同角色围绕

同一业务对象执行不同操作时，如何在保证数据共享的同时实现严格的权限边界控制与状态一致性。

3.2.2 功能需求分析

结合业务场景，系统的功能需求主要包括以下几个方面。

第一，用户认证与账号管理需求。系统应支持多角色用户登录、身份校验、角色识别和账号维护，确保不同角色在合法身份前提下访问相应资源。

第二，订单管理需求。顾客能够提交订单，销售管理员完成审核后，订单进入后续仓储、生产和质检处理流程。系统需要完整记录订单创建、审核、执行和交付状态。

第三，库存管理需求。系统需支持成品与原材料库存管理、出入库记录维护、库存数量查询以及库存预警判定，以支撑仓库作业和计划联动。

第四，生产协同需求。当仓库发现成品库存不足以满足订单交付时，系统应能够触发生产计划并将任务传递给生产管理员；生产完成后需进入质检流程，合格后再由仓库入库和发货。

第五，采购供应协同需求。原材料不足时，系统需支持采购申请、采购订单生成、供应商接单发货以及仓库收货入库等操作，从而保证生产连续性。

第六，质检与质量追溯需求。系统应记录关键质量检验信息，并支持顾客或内部人员基于订单信息查询质量处理链路，提升业务透明度。

第七，实时通知与周计划生成功能需求。系统应在关键业务节点主动向相关角色推送消息，并结合库存、安全库存、历史消耗与在途数据生成生产和采购周计划，辅助业务决策。

3.2.3 非功能需求分析

除功能需求外，系统还应满足以下非功能需求。

在安全性方面，系统应具备身份认证、访问控制和敏感操作保护能力，防止越权访问和非法操作。在可用性方面，系统界面应保持操作流程清晰、反馈及时，便于不同岗位快速上手。在一致性方面，系统应确保订单状态、库存数量、生产记录和采购状态在多角色协同处理中保持同步。在可维护性方面，系统应采用清晰的分层结构，便于后续模块扩展和规则调整。在可扩展性方面，系统应支持后续对接移动端、报表分析模块或更细粒度的计划优化模块。

3.3 整体架构设计

3.3.1 系统总体架构

系统总体架构采用典型的前后端分离模式，由前端表示层、后端业务层、数据持久层和消息通信层构成。前端负责提供面向不同角色的业务页面与交互入口；后端负责认

证鉴权、流程控制、业务规则校验和数据整合；数据持久层承担结构化业务数据的存储；消息通信层负责关键业务事件的实时通知。

在该架构下，顾客、企业内部各岗位以及供应商均通过浏览器访问系统，前端通过 HTTP 接口与后端进行业务交互，并通过 WebSocket 通道接收实时通知。整体架构既满足多角色统一访问需求，也能够通过服务层将复杂业务规则集中管理，避免将流程判断分散在前端页面中。

3.3.2 前后端分层设计

后端按照控制层、服务层、持久层进行分层设计。控制层负责接收前端请求、进行参数校验并返回标准化响应；服务层封装业务规则、状态流转和事务控制，是系统核心逻辑的承载层；持久层依托 JPA 完成实体管理和数据访问。前端则按照页面层、组件层、接口调用层进行组织，页面层负责业务视图组合，组件层负责通用交互单元复用，接口调用层统一处理请求发送、令牌附带和异常响应。

该分层设计使系统具有较好的职责分离特征。例如，订单审核是否能够进入仓库处理、库存不足时是否触发生产、质检不合格是否允许入库等判断逻辑，均由服务层统一完成，从而保证不同客户端入口下的业务规则一致性。

3.3.3 权限控制设计

权限控制设计遵循“先认证、后授权、按角色分域访问”的原则。系统登录后由后端签发 JWT，后续请求中由安全过滤器解析令牌并恢复用户身份。资源访问控制采用路径级与方法级相结合的方式，对不同角色开放不同接口权限。例如，顾客只允许执行下单、订单查询和追溯查询等操作；销售管理员可审核订单但无权处理库存；仓库管理员可维护库存与发货但无权配置系统账号；供应商仅可访问与自身采购订单相关的业务页面。

此外，权限控制不仅体现在页面是否显示，更体现在服务层业务校验中。即便前端页面进行了入口限制，后端仍需根据当前用户角色和业务状态进行二次校验，从而避免通过构造请求绕过前端控制。

3.4 业务流程设计

3.4.1 客户下单与销售审核流程

顾客在系统中提交订单后，订单首先进入待审核状态。销售管理员对订单内容、交付要求和业务合法性进行审查，审核通过后订单进入仓库处理环节，审核不通过则返回相应处理结果。该流程的设计目的在于保证进入执行阶段的订单具备业务有效性，并由销售角色承担订单入口把关职责。

3.4.2 仓储发货与库存流转流程

仓库管理员接收已审核订单后，首先校验成品库存是否满足发货需求。若库存充足，则执行出库与发货操作，更新库存数量和库存流水；若库存不足，则将缺口信息反馈至生产协同流程。对于已经完成生产并经质检合格的产品，仓库需执行入库操作后再根据订单安排发货。该流程体现了库存作为订单履约关键中间状态的重要作用。

3.4.3 生产计划、生产执行与质检流程

当订单无法直接由现有成品库存满足时，系统根据缺口数量生成生产需求。生产管理员据此制定或接收生产任务，组织生产执行并记录生产过程。生产完成后，产品进入质检环节，由质检管理员记录检验结果。检验合格则允许入库并支持后续发货，不合格则需进入异常处理或返工路径。该流程保证了“先生产、后质检、再入库”的业务顺序，避免未检产品直接进入库存。

3.4.4 原材料采购与供应商协同流程

采购流程既可以由采购管理员主动发起，也可以由库存预警或周计划联动触发。当原材料数量低于安全库存或周计划预测出未来物料缺口时，系统生成采购申请。采购管理员审核后形成采购订单，并向对应供应商发送通知。供应商确认接单后执行发货，仓库在收货后完成入库，必要时可配合质检管理员进行来料质检。该流程强化了企业内部采购部门与外部供应商之间的业务协同。

3.4.5 库存预警与周计划生成流程

库存预警流程通过比较当前库存、在途数量和安全库存阈值，对成品和原材料的潜在缺口进行识别。一旦触发预警，系统向相关角色推送通知，并为周计划生成提供输入依据。周计划生成则综合订单需求、库存余量、历史消耗、在途物料和产品物料清单信息，给出生产周计划与采购周计划建议。该流程的设计目标是将被动响应转化为主动预判。

3.4.6 质量追溯流程

质量追溯流程围绕订单号或产品关联标识展开，将订单信息、生产记录、质检记录和出入库记录进行贯通。当顾客发起质量追溯查询时，系统可展示相应订单在生产、检验和交付过程中的关键处理记录。该流程有助于提升问题定位效率，并增强顾客对企业交付过程透明度的信任。

3.5 数据库设计

3.5.1 概念结构设计

数据库设计围绕业务主对象展开，主要包含用户、角色、产品、订单、订单明细、库存、库存流水、生产计划、生产记录、采购申请、采购订单、供应商、原材料、质检记录和消息通知等实体。各实体通过主外键关系构成完整业务链路。例如，订单与订单明细之间为一对多关系，产品与库存之间存在对应关系，生产计划与订单或库存缺口之间存在关联，采购订单与供应商之间存在从属关系，质检记录则与生产记录或来料记录相关联。

3.5.2 主要数据表设计

表 3-2 系统主要数据表设计

表名	主要字段	功能说明
user	id、username、password、status	存储系统用户基础信息与登录凭据
role	id、role_name、description	存储角色定义与权限归属关系
product	id、product_name、specification、unit	存储产品基础信息
sales_order	id、order_no、customer_id、status、create_time	存储客户订单主信息与状态
sales_order_item	id、order_id、product_id、quantity	存储订单明细项
inventory	id、item_type、item_id、current_qty、safe_qty	存储成品或原材料库存信息
inventory_record	id、inventory_id、change_type、qty、operator	存储出入库流水记录
production_plan	id、plan_no、product_id、plan_qty、status	存储生产计划信息
purchase_order	id、purchase_no、supplier_id、status、arrival_status	存储采购订单与到货状态
quality_record	id、biz_type、biz_id、result、remark	存储质检结果与追溯依据
message_notice	id、receiver_id、message_type、is_read	存储实时通知留痕记录

表 3-2给出了系统主要数据表设计。通过上述表结构，系统能够对订单执行、库存变动、生产任务、采购协同、质检记录和消息通知进行统一管理。

3.5.3 关键实体关系分析

在关键实体关系中，用户与角色之间为多对多关系，用于支持同一用户持有一个或多个角色；订单与订单明细之间为一对多关系，确保一个订单可以包含多个产品项；产品与库存之间通过业务类型区分成品库存与原材料库存；生产计划通常关联特定产品和触发来源；采购订单与供应商之间为多对一关系；质检记录通过业务标识与生产或来料

过程相关联；消息通知则与接收用户和业务事件绑定。

实体关系设计的核心目标在于支撑系统业务闭环。一方面，各实体需能独立维护自身状态和属性；另一方面，不同实体之间必须通过关联关系反映真实业务链路，以保证查询、追溯和状态传播的正确性。这一设计为后续详细实现提供了数据层基础。

4 系统详细设计与实现

4.1 开发环境与工具

本系统采用前后端分离方式进行开发与部署。后端以 Java 17 和 Spring Boot 2.7 为核心开发框架，借助 Maven 完成依赖管理和项目构建；前端采用 Vue 3 与 Vite 构建单页应用；数据层使用 MySQL 管理业务数据；开发过程中通过浏览器开发者工具和接口调试工具对请求响应与页面交互进行辅助验证。系统开发环境如表 4-1所示。

表 4-1 系统开发环境与工具

类别	名称	说明
操作系统	Windows	系统开发与论文撰写环境
后端语言	Java 17	后端业务逻辑开发
后端框架	Spring Boot 2.7	RESTful 服务构建与模块整合
安全框架	Spring Security	认证授权与访问控制
持久化技术	Spring Data JPA	实体映射与数据库访问
数据库	MySQL	业务数据持久化存储
前端框架	Vue 3	页面构建与交互实现
构建工具	Vite、Maven	前后端工程构建与依赖管理
通信技术	SockJS、STOMP	实时消息推送与订阅
HTTP 调用	Axios	前后端接口访问封装

4.2 各功能模块实现

4.2.1 用户认证与账号管理模块实现

用户认证与账号管理模块是系统运行的入口。后端通过 Spring Security 构建安全过滤链，在登录请求进入系统后，由认证管理器完成用户名和密码校验，认证成功后生成 JWT 返回前端。前端在登录成功后保存令牌，并在 Axios 请求拦截器中统一附加到后续请求头，实现多页面访问下的身份延续。后端在每次请求到达时解析令牌并恢复用户身份信息，将角色集合注入安全上下文，从而支撑后续接口鉴权。

账号管理功能主要面向系统管理员。管理员可以维护用户基本信息、角色归属和账号状态。为防止越权，涉及账号新增、停用和角色变更等操作均在后端服务层进行角色校验，并对关键更新操作记录审计信息。前端页面则通过角色控制决定功能入口是否可见，实现页面限制与接口限制双重防护。

4.2.2 顾客下单与订单管理模块实现

顾客下单模块以前端订单表单为载体，支持产品选择、数量填写、订单提交和历史订单查询。前端在提交前对订单项完整性进行基础校验，然后通过 Axios 向后端订单接口发送请求。后端控制层接收订单数据后，调用服务层完成订单主表与订单明细表写

入，并初始化订单状态为待审核。

订单管理模块贯穿系统多个角色。顾客可查看订单处理进度，销售管理员可执行审核，仓库管理员可基于订单触发出库或生产联动。为保证业务一致性，订单状态不允许任意跳转，而必须按照既定流程由服务层统一推进。这样能够避免前端直接修改状态造成的链路断裂问题。

4.2.3 销售管理模块实现

销售管理模块的核心在于订单审核与销售记录维护。销售管理员进入审核页面后，可查看待审核订单详情，对订单有效性、数量合理性和交付约束进行确认。审核操作提交后，后端服务层根据结果更新订单状态，并生成对应的销售处理记录。当审核通过时，系统进一步向仓库管理员推送待处理通知；当审核不通过时，则将结果反馈给顾客。

从实现机制看，销售管理模块通过“订单主表状态更新 + 销售处理记录写入 + 消息通知推送”的组合方式完成业务闭环，既保留了审核留痕，也为下游角色提供了明确的处理依据。

4.2.4 仓库管理与库存预警模块实现

仓库管理模块包含库存查询、入库、出库、发货和库存预警等功能。系统在数据库中维护成品和原材料库存实体，并通过库存流水表记录每次数量变化的原因、数量和操作人员。仓库管理员在执行入库、出库或发货操作时，前端向后端提交库存变更请求，后端在事务中同步更新当前库存数量与流水记录，以保证结果一致。

库存预警模块通过定期计算或业务触发检查库存状态。当当前数量叠加在途数量后仍低于安全库存阈值时，系统生成预警信息并写入消息表，同时通过实时通知机制推送给仓库管理员和采购管理员。该模块的实现重点在于将库存风险识别从人工经验判断转变为规则化处理，为生产和采购协同提供前置信号。

4.2.5 生产管理模块实现

生产管理模块负责承接库存不足或周计划下发后的生产需求。后端服务根据订单缺口或计划结果生成生产计划记录，并分配给生产管理员。生产管理员可在前端页面中查看待执行任务、填报执行进度和维护生产结果。生产结束后，系统将对应任务状态更新为待质检，并触发质检通知。

在实现层面，生产管理模块的关键在于对任务状态进行严格控制。生产计划、生产执行、生产完成和待质检等状态由服务层统一推进，并与订单履约需求和库存状态联动，从而保证生产活动不是孤立存在，而是服务于订单交付与库存补充。

4.2.6 采购与供应商管理模块实现

采购与供应商管理模块支持采购申请、采购订单生成、供应商接单、发货反馈和仓库收货入库等业务。采购管理员在前端页面中可查看由库存预警或周计划触发的采购建议，也可手动发起采购申请。申请经确认后由后端生成采购订单，并将订单推送给对应供应商。供应商登录后可查看待处理订单，执行接单和发货确认操作。仓库管理员在货物到达后完成收货入库，必要时可与质检模块联动进行来料检验。

该模块的实现重点在于企业内部与外部供应商之间的信息贯通。系统不仅记录采购单据本身，还通过状态更新与消息通知保持采购管理员、供应商和仓库管理员之间的信息同步，从而降低人工跟踪成本。

4.2.7 质检与质量追溯模块实现

质检模块主要用于处理生产完成后的成品质检以及必要的来料质检。质检管理员在系统中查看待检任务，录入检验结果、检验时间和备注信息。若检验合格，系统允许对应产品入库或进入后续发货流程；若检验不合格，则阻断后续流转并保留异常记录。

质量追溯模块则基于订单号或关联业务标识整合订单信息、生产记录、质检记录和库存流转记录。顾客在追溯页面输入订单信息后，后端按链路查询相关记录并返回结果。通过该机制，系统能够对关键业务环节形成可查询、可核验的数据链条，提高交付透明度。

4.2.8 实时消息通知模块实现

实时消息通知模块基于 WebSocket/STOMP 构建。后端在订单审核通过、库存预警触发、采购订单下发、供应商发货、生产任务生成、质检完成等业务节点，向指定主题或用户消息通道发送通知。前端在用户登录后建立 WebSocket 连接，并订阅与当前角色相关的消息主题。消息到达后，前端可刷新待办数量、展示通知列表或更新页面局部状态。

为了保证消息可追溯，系统不仅进行即时推送，还将通知内容写入消息通知表。这样即使用户在离线状态下未实时接收消息，也可在下次登录时查看历史通知，从而兼顾实时性与留痕性。

4.2.9 周计划智能生成模块实现

周计划生成模块并非依赖复杂智能算法，而是基于业务规则对未来一周的供需关系进行计算。后端服务首先汇总订单需求、当前库存、在途数量、安全库存和历史消耗数据，再结合产品与原材料之间的 BOM 关系估算未来一周的生产量和采购量建议。系统生成结果后，生产管理员和采购管理员可在对应页面中查看并作为业务决策参考。

在实现链路上，该模块通过计划服务统一读取订单、库存、采购和产品配置数据，

完成规则计算后将结果写入生产周计划表与采购周计划表，并向相关角色发送通知。该设计体现了系统从事务处理向辅助决策延伸的工程价值。

5 核心功能实现

5.1 基于角色的访问控制与身份认证实现

本系统面向多角色协同场景，若缺乏清晰的认证授权机制，容易出现数据越权访问和职责冲突问题。因此，系统在安全设计上采用“登录认证 + 令牌校验 + 角色授权 + 业务再校验”的分层控制策略。用户通过账号密码完成认证后，后端生成 JWT，令牌中包含用户标识和角色声明；后续请求到达后，由安全过滤器解析令牌并恢复身份上下文。接口级权限控制根据角色限制访问入口，服务层再结合业务状态进行二次校验。

该机制的工程价值主要体现在两方面。其一，采用 JWT 的无状态认证方式，可以降低服务端会话维护成本，更适合前后端分离架构；其二，基于角色的访问控制使系统能够在统一平台上兼容多类用户，而无需为每类角色单独构建隔离系统。以订单处理为例，顾客只能创建和查询订单，销售管理员只能审核订单，仓库管理员只能在审核通过后处理出入库，角色边界清晰且具备可验证性。

5.2 订单—仓储—生产—质检协同流程实现

订单—仓储—生产—质检协同流程是本系统最具代表性的闭环业务链路。系统并未将订单处理、库存管理和生产执行视为彼此独立的模块，而是通过订单状态、库存状态和任务状态的联动，使其形成连续的流程驱动机制。顾客下单后，订单进入销售审核；审核通过后，仓库根据库存情况决定是直接发货还是触发生产；生产完成后进入质检；质检合格后再由仓库入库并完成发货。

该流程的实现难点在于状态控制与跨模块一致性。为解决该问题，系统采用服务层集中控制状态流转的方法，将订单、库存、生产和质检相关更新置于统一事务边界内处理。这样既避免了状态跳转混乱，也确保了例如“未质检产品不可入库”“库存不足才可触发生产”等规则能够被严格执行。该设计体现了以流程驱动替代人工口头协同的工程思路。

5.3 采购—供应商—仓库协同流程实现

采购协同流程的目标是保证原材料供应连续性，并使企业内部采购部门与外部供应商之间的信息传递更加及时。系统在库存预警、周计划计算或人工申请触发后生成采购需求，采购管理员确认后形成采购订单。随后，系统向供应商推送待处理通知，供应商登录后执行接单与发货反馈。货物到达企业后，由仓库管理员完成收货入库，并视需要结合质检管理员完成来料检验。

该流程的实现重点在于多主体状态同步。采购订单在创建、确认、发货、到货和入

库等节点均会更新状态，并通过消息通知同步给相关角色。这样可显著降低电话、即时通信软件等非结构化沟通方式造成的信息遗漏风险。同时，系统保留各处理节点记录，为后续供应商履约评估和采购追踪提供数据基础。

5.4 库存预警与周计划自动生成算法实现

库存预警与周计划生成的设计目标并非追求复杂的预测模型，而是在企业现有业务数据基础上构建可落地的规则计算机制。库存预警首先针对成品和原材料分别计算预计可用库存，当预计库存低于安全库存阈值时即触发预警。对于产品 p 在某一计划周期内的预计库存，可表示为

$$I_p^{\mathrm{proj}} = I_p^{\mathrm{cur}} + I_p^{\mathrm{in}} - D_p^{\mathrm{ord}} - D_p^{\mathrm{plan}} \tag{5-1}$$

其中，I_p^{cur} 表示当前库存，I_p^{in} 表示在途入库数量，D_p^{ord} 表示已确认订单需求，D_p^{plan} 表示计划期间额外消耗量。当 I_p^{proj} 小于安全库存时，系统生成预警信息。

在周计划生成阶段，系统进一步综合需求量、安全库存和现有供给，计算建议生产量。其基本思想可表示为

$$Q_p^{(w)} = \max\left(0, \hat{D}_p^{(w)} + S_p^{(w)} - I_p^{\mathrm{cur}} - I_p^{\mathrm{in}}\right) \tag{5-2}$$

其中，$\hat{D}_p^{(w)}$ 表示未来一周需求预测值，$S_p^{(w)}$ 表示安全库存目标，$Q_p^{(w)}$ 表示建议生产量。若产品存在物料清单，则可进一步估算原材料需求量

$$M_m^{(w)} = \sum_{p=1}^{n} Q_p^{(w)} \times B_{pm} \tag{5-3}$$

其中，B_{pm} 表示产品 p 对原材料 m 的单位消耗量。通过上述规则，系统能够生成生产周计划与采购周计划建议。其优点在于实现成本低、可解释性强、便于业务人员理解和修正。

为了保证规则计算流程清晰，系统在计划服务中采用如下处理步骤。

算法 5-1: 周计划自动生成流程

Input: 订单需求集合、库存数据、在途数据、安全库存阈值、BOM 数据

Output: 生产周计划、采购周计划

```
汇总未来一周已确认订单需求;
计算产品预计可用库存;
foreach 产品 do
    若预计库存低于安全库存，则计算建议生产量;
end
根据建议生产量和 BOM 关系换算原材料需求;
foreach 原材料 do
    若预计物料库存低于阈值，则生成采购建议量;
end
写入生产周计划与采购周计划记录;
向生产管理员和采购管理员发送计划通知;
```

5.5 实时消息推送与状态同步实现

在多角色协同系统中，若业务状态变化不能及时传递给相关岗位，即使系统具备完整功能，也难以真正提升协同效率。为此，本系统采用 WebSocket/STOMP 构建实时消息机制，将订单审核、库存预警、采购下发、生产任务生成和质检结果等关键事件转换为可订阅消息。后端以业务事件为触发点向指定用户或角色主题推送通知，前端在收到消息后更新页面待办项和状态展示。

该机制的关键价值在于将原本被动查询式的信息获取转变为主动推送式的状态同步。例如，销售审核通过后，仓库无需周期性刷新页面等待新任务；质检完成后，仓库可立即获知是否允许入库；采购订单生成后，供应商可以直接接收待处理提醒。由此可见，实时消息机制不仅改善了用户体验，更增强了业务流程的连续性。

5.6 质量追溯功能实现

质量追溯功能的核心在于将订单、生产、质检和库存流转记录进行贯通，并围绕统一的业务标识建立查询链路。系统在订单生成时即建立基础追踪主线，在后续生产任务、质检结果和出入库记录中保留与订单或产品批次相关的关联字段。当顾客或管理人员发起追溯查询时，后端服务依据订单号或业务标识聚合相关记录，并按时间顺序返回处理链路。

从工程实现角度看，质量追溯并不依赖额外复杂模型，而是依赖业务处理过程中对关键数据节点的完整留痕。只要系统能够保证生产记录、质检记录和库存记录的时序性

与关联性，就能够在问题发生后快速定位处理节点和责任环节。因此，质量追溯既是系统透明度的重要体现，也是后续质量管理优化的数据基础。

6 系统测试

6.1 测试环境与工具

为验证系统设计与实现的正确性，本文围绕认证授权、订单处理、库存管理、生产流转、采购协同、质量追溯与实时通知等功能开展测试。测试环境覆盖后端运行环境、数据库环境和前端访问环境，具体配置如表 6-1所示。

表 6-1 系统测试环境

类别	配置项	说明
操作系统	Windows	系统部署与测试运行环境
JDK	Java 17	后端运行环境
构建工具	Maven、Node.js	前后端依赖安装与项目构建
数据库	MySQL	业务数据存储与测试数据准备
前端运行环境	Vue 3、Vite	页面访问与交互验证
浏览器	Chrome 或 Edge	功能页面与实时通知验证

测试过程中采用模块化验证思路：先对关键服务进行单元级检查，再对跨角色流程进行功能测试，以确保系统既满足局部业务规则，又能够在整体流程上保持贯通。

6.2 单元测试

单元测试主要面向后端关键服务层逻辑，重点验证认证模块、订单流程、生产流程、采购流程、质检流程以及周计划生成服务等是否符合预期。测试内容主要包括用户登录后令牌生成与解析是否正确、订单状态转换是否符合约束、库存更新是否与库存流水一致、生产完成后是否正确进入待质检状态、采购订单状态变化是否准确，以及周计划生成结果是否能够正确识别缺口并给出建议。

表 6-2 关键单元测试内容

测试对象	测试内容	预期结果
认证服务	登录认证、令牌生成、令牌解析	用户身份恢复正确，非法令牌被拒绝
订单服务	订单创建、审核、状态推进	状态按既定流程流转，不允许越级跳转
库存服务	入库、出库、库存流水更新	当前库存与流水记录保持一致
生产服务	生产计划生成、任务完成后转待检	生产任务状态更新正确，自动触发质检流程
采购服务	采购申请、采购单生成、到货处理	采购状态与供应商反馈保持同步
计划服务	库存预警判断、周计划计算	能识别供需缺口并生成合理建议

单元测试结果表明，系统关键服务层逻辑能够按照设计规则稳定运行，尤其是在状态控制和权限校验方面能够较好地保证业务一致性。

6.3 功能测试

功能测试以实际业务场景为中心，重点验证多角色协同流程是否通畅。测试覆盖登录认证、顾客下单、销售审核、库存查询、仓库发货、库存不足触发生产、生产完成进入质检、采购协同、供应商发货、质量追溯查询和实时消息通知等功能。功能测试结果如表 6-3所示。

表 6-3 主要功能测试结果

测试场景	测试要点	结果
登录认证	不同角色登录后进入对应功能范围，非法访问被拦截	通过
订单处理	顾客下单后进入销售审核，审核结果可回传	通过
库存管理	出入库操作后库存数量与流水同步更新	通过
生产流转	库存不足时触发生产，生产完成后进入待质检	通过
采购协同	采购订单可下发给供应商，供应商可反馈发货状态	通过
质量追溯	可依据订单号查询生产与质检链路信息	通过
实时通知	审核、预警、计划与质检结果可实时推送	通过
周计划生成	可生成生产与采购建议并通知相关角色	通过

从功能测试情况看，系统能够支撑主要业务场景的连续处理，不同角色之间的协作关系在系统中得到了较为清晰的体现。

6.4 测试结果分析

综合单元测试与功能测试结果可以看出，本文设计并实现的企业供应链协同管理系统能够基本满足预期设计要求。系统在认证授权、订单处理、库存流转、生产协同、采购协同、质检留痕、质量追溯和实时通知等方面均表现出较好的功能完整性。尤其是在多角色协同闭环和状态一致性方面，系统通过统一的服务层规则控制和消息同步机制，有效降低了人工流转过程中信息延迟或遗漏的风险。

但从测试结果来看，系统仍存在进一步优化空间。首先，周计划生成目前主要基于规则计算，对需求波动的适应能力仍有限；其次，实时通知机制已能满足一般业务协同需要，但在更大并发量下仍需进一步评估性能表现；再次，当前系统主要面向 Web 端访问，移动端适配能力尚有待增强。总体而言，系统已具备较好的工程可用性，但在计划智能化、性能优化和终端扩展方面仍有继续提升的空间。

7 总结与展望

7.1 工作总结

本文围绕企业供应链协同管理场景，完成了基于 Spring Boot 与 Vue 3 的企业供应链协同管理系统的分析、设计与实现工作。针对传统管理模式下存在的信息分散、流程衔接不畅、库存状态不透明和质量追溯链条不完整等问题，本文从业务协同需求出发，设计了覆盖顾客、销售管理员、仓库管理员、生产管理员、采购管理员、供应商、质检管理员和系统管理员等多角色的统一业务平台。

在系统设计方面，本文完成了需求分析、总体架构设计、权限控制设计、业务流程设计和数据库设计，明确了订单驱动下的销售审核、库存处理、生产执行、采购供应、质检留痕和质量追溯等关键业务链路。在系统实现方面，本文基于 Spring Boot 构建后端服务，结合 Spring Security 与 JWT 实现认证授权，采用 Vue 3、Vite 与 Axios 构建前端交互界面，以 MySQL 和 JPA 管理业务数据，并利用 WebSocket/STOMP 实现实时消息推送。

在核心功能层面，本文重点实现了基于角色的访问控制机制、订单—仓储—生产—质检协同流程、采购—供应商—仓库协同流程、库存预警与周计划生成机制、实时消息同步机制以及质量追溯功能。测试结果表明，系统能够较好地支撑企业供应链业务的基本运行需求，在流程贯通性、数据一致性和业务可追溯性方面具有较好的应用价值。

7.2 研究不足与未来展望

尽管本文完成了系统的主要设计与实现工作，但仍存在一定不足。首先，周计划生成目前主要基于规则和阈值计算，对复杂需求波动、季节性变化和多约束排程问题的适应能力仍较弱。其次，系统当前更偏重业务流程协同，在经营分析、数据可视化和决策辅助方面的深度仍有提升空间。再次，本文实现主要面向 Web 端，尚未针对移动端场景进行专门优化，在现场作业和移动审批等方面仍不够完善。此外，当前系统采用单体架构实现，虽然便于开发与维护，但在更大规模业务场景下的弹性伸缩和模块解耦能力仍有待增强。

后续研究可从以下几个方向展开。其一，可引入更精细的需求预测方法和多因素计划优化策略，增强生产与采购计划生成的准确性与适应性。其二，可进一步完善可视化分析功能，围绕订单履约、库存周转、供应商响应和质量问题分布等内容构建管理看板。其三，可扩展移动端应用，提高仓储、质检和采购审批等场景下的便捷性。其四，可探索系统的微服务化改造，增强模块独立部署能力和高并发场景下的系统稳定性。由此可见，企业供应链协同管理系统仍具有较大的优化空间和工程应用前景。

致谢

本文的完成离不开许多老师、同学和家人的帮助与支持。在论文选题、系统设计、实现过程以及论文撰写的各个阶段，指导教师李运发老师给予了细致的指导与耐心的帮助。老师在研究思路梳理、论文结构安排以及技术细节把控等方面提出了许多具有启发性的意见，使本文得以逐步完善。在此谨向李老师致以诚挚的感谢。

同时，感谢学院各位任课教师在本科阶段所传授的专业知识与工程实践方法，为本文的完成奠定了扎实基础。感谢同学们在系统需求分析、技术交流与论文修改过程中提供的帮助与建议，使本文在内容表达与结构组织上更加严谨。

最后，感谢家人在学习和生活中给予的理解、支持与鼓励。正是这些持续的关心与陪伴，使本人能够以较为稳定的状态完成毕业设计与论文撰写工作。

参考文献

[1] Christopher M. Logistics & Supply Chain Management[M]. 5th ed. Harlow: Pearson, 2016.

[2] Lee H L. The Triple-A Supply Chain[J]. Harvard Business Review, 2004, 82(10): 102–112.

[3] Gunasekaran A, Patel C, Tirtiroglu E. A framework for supply chain performance measurement[J]. International Journal of Production Economics, 2004, 87(3): 333–347.

[4] Monk E, Wagner B J. Concepts in Enterprise Resource Planning[M]. 4th ed. Boston: Cengage Learning, 2012.

[5] 中华人民共和国国家质量监督检验检疫总局, 中国国家标准化管理委员会. GB/T 22005-2009 饲料和食品链的可追溯性体系设计与实施通用原则和基本要求 [S]. 2009.

[6] Fielding R T. Architectural Styles and the Design of Network-Based Software Architectures[D]. Irvine: University of California, Irvine, 2000.

[7] VMware, Inc.. Spring Boot Reference Documentation[EB/OL]. 2024 [2026-04-20]. https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/.

[8] VMware, Inc.. Spring Security Reference[EB/OL]. 2024 [2026-04-20]. https://docs.spring.io/spring-security/reference/.

[9] Jones M, Bradley J, Sakimura N. RFC 7519: JSON Web Token (JWT)[EB/OL]. IETF, 2015 [2026-04-20]. https://www.rfc-editor.org/rfc/rfc7519.

[10] Vue.js Team. Vue.js Guide[EB/OL]. 2024 [2026-04-20]. https://vuejs.org/guide/introduction.html.

[11] Vite Team. Vite Guide[EB/OL]. 2024 [2026-04-20]. https://vite.dev/guide/.

[12] Fette I, Melnikov A. RFC 6455: The WebSocket Protocol[EB/OL]. IETF, 2011 [2026-04-20]. https://www.rfc-editor.org/rfc/rfc6455.

[13] STOMP Protocol Community. STOMP Specification Version 1.2[EB/OL]. 2012 [2026-04-20]. https://stomp.github.io/stomp-specification-1.2.html.

[14] Oracle Corporation. MySQL 8.0 Reference Manual[EB/OL]. 2024 [2026-04-20]. https://dev.mysql.com/doc/refman/8.0/en/.

[15] VMware, Inc.. Spring Data JPA Reference Documentation[EB/OL]. 2024 [2026-04-20]. https://docs.spring.io/spring-data/jpa/reference/.